Model card — movement recognition and impact (fall) detection

Generated 2026-10-17. Every number in this document was produced by the scripts in `ml/` and read back out of the JSON artefacts in `ml/outputs/har/`, not typed in by hand. Where a claim cannot be supported by the data, this card says so.

---

1. What these models are

The app has **two supervised models**, both shipped inside the APK and both evaluated on the phone:

#	Model	Question it answers	Labels come from
A	**Activity recognition**	"Is this person walking, walking upstairs, walking downstairs, sitting, standing or lying down?"	Real human labels — UCI HAR's own annotations
B	**Impact / fall-signature detector**	"Does this window contain the acceleration signature of a hard impact followed by stillness?"	Real recordings, with a **synthesised impact injected** to create the positive class

Both are ordinary **scikit-learn random forests** exported to JSON and walked on the device by hand-written Kotlin. There is no TensorFlow, no TFLite, no PyTorch and no network call at inference time. This is a deliberate size and reproducibility choice, and §7 proves the Kotlin walker agrees with scikit-learn.

What this is not

- **Not a medical device.** It does not diagnose a fall, a syncope, a seizure or an injury.
- **Not a real-fall detector.** UCI HAR contains no falls. Model B is trained on a *simulated* impact signature and its metrics measure that simulation (§5).
- **Not a lot-of-data model.** 30 subjects in a controlled lab protocol is small. §8 lists what that costs.

The app never raises a fall alert on the model's opinion alone — see §6.

---

2. Data

Source: UCI HAR (Human Activity Recognition Using Smartphones), dataset id `uci-240`, registered in `DATASETS.pdf`. The GeoLife GPS trajectories in the same document are used by the separate trajectory-anomaly work, **not** by these models: GeoLife has no accelerometer or gyroscope channels, so it cannot train a movement model. Saying otherwise would be the easiest lie in this project.

Property	Value
Windows	7352 train / **2947 test**
Window length	128 samples @ 50 Hz = **2.56 s**
Raw channels per split	9 (`total_acc_x/y/z`, `body_acc_x/y/z`, `body_gyro_x/y/z`)
Feature vectors (561-dim)	561 per window, names from the dataset's own `features.txt`
Train subjects	21 — 1, 3, 5, 6, 7, 8, 11, 14, 15, 16, 17, 19, 21, 22, 23, 25, 26, 27, 28, 29, 30
Test subjects	9 — 2, 4, 9, 10, 12, 13, 18, 20, 24
Subject overlap	**none** (`subject_overlap: []`)

The split is the dataset's official one and it is **subject-disjoint**: no person appears in both sides. That is the only honest way to report accuracy here, because windows from one person's recording are highly correlated and a random window split would leak.

Sanity checks recorded in `ml/outputs/har/clean/har_dataset_meta.json`:

- `train_windows = 7352`, `test_windows = 2947`, `feature_count = 561`, `raw_window_length = 128`
- `raw_windows_match_features = true`, `feature_vectors_match_names = true`
- `train_non_finite = 0`, `test_non_finite = 0`
- `raw_gravity_sanity`: mean total-acceleration magnitude **1.0323 g** (train) / **1.0328 g** (test) — a real measurement centring on gravity, not a constant or a fabrication
- `body_gyro_train_rms = 0.4068`, `total_acc_x_train_min_max = [-0.4666, 2.1976]`

---

3. The 24 features the app actually computes

The phone cannot compute all 561 UCI features in real time (they include FFT-derived energy bands and entropy terms). The app computes a **24-feature, time-domain-only** subset that mirrors `MotionFeatures.kt` exactly, in the same order. Accelerometer samples are converted to g by dividing by `9.80665 m/s²` so the units match the dataset.

#	Feature	Meaning
1–8	`acc_mag_mean`, `acc_mag_std`, `acc_mag_min`, `acc_mag_max`, `acc_mag_range`, `acc_mag_energy`, `acc_mag_peak_jerk`, `acc_mag_zcr`	statistics of the acceleration magnitude: level, spread, extremes, spread, mean-square, largest sample-to-sample jump, zero-crossing rate
9–11	`acc_x_mean`, `acc_y_mean`, `acc_z_mean`	mean per-axis acceleration — the **gravity direction**, i.e. how the phone is held
12–14	`acc_x_std`, `acc_y_std`, `acc_z_std`	per-axis movement intensity
15–17	`acc_xy_corr`, `acc_xz_corr`, `acc_yz_corr`	inter-axis correlation
18–21	`gyro_mag_mean`, `gyro_mag_std`, `gyro_mag_max`, `gyro_mag_energy`	rotation-rate magnitude statistics
22–24	`gyro_x_std`, `gyro_y_std`, `gyro_z_std`	per-axis rotation intensity

Adding features 9–11 (the gravity direction) is what made the on-device model work: for a phone that is merely *held* differently, the acceleration component (18) and the axis correlations (15–17) are almost unchanged, while the mean per-axis values move a lot. During development the same forest built on the 21 features without them scored **0.7581** against the shipped set's **0.8351**. That ablation was run while choosing the feature set and is recorded in `docs/progress_log.md`; it is not regenerated by `har_train.py`, so treat it as a development measurement rather than a shipped artefact.

Why 24 is enough — measured, not asserted:

Features used	1	2	5	12	20	30	50	80	120	200	561
Accuracy	0.397	0.563	0.612	0.768	0.804	0.826	0.874	0.889	0.907	0.918	0.925

Past roughly 50 features the curve flattens; the 561-feature model buys **+0.09** accuracy for 23× the computation. `fig_har_05_accuracy_vs_features.png` plots this.

---

4. Model A — activity recognition (real labels)

Configuration: `RandomForestClassifier`, `n_estimators=32`, `max_depth=8`, 24 features, fixed `random_state`. Trained on 7352 windows from 21 subjects, evaluated once on 2947 windows from 9 unseen subjects.

Headline metrics (held-out, subject-disjoint)

Model	Features	Trees	Accuracy	Macro-F1
Report reference	561	300	**0.9287**	**0.9271**
On-device (shipped)	**24**	**32**	**0.8351**	**0.8333**
Single decision tree (baseline)	24	1	0.7967	0.7943

The shipped model is 0.0936 behind the 561-feature reference and 0.0384 ahead of a single tree. Both comparisons are reported because either one alone is misleading.

Per-class results — shipped 24-feature model

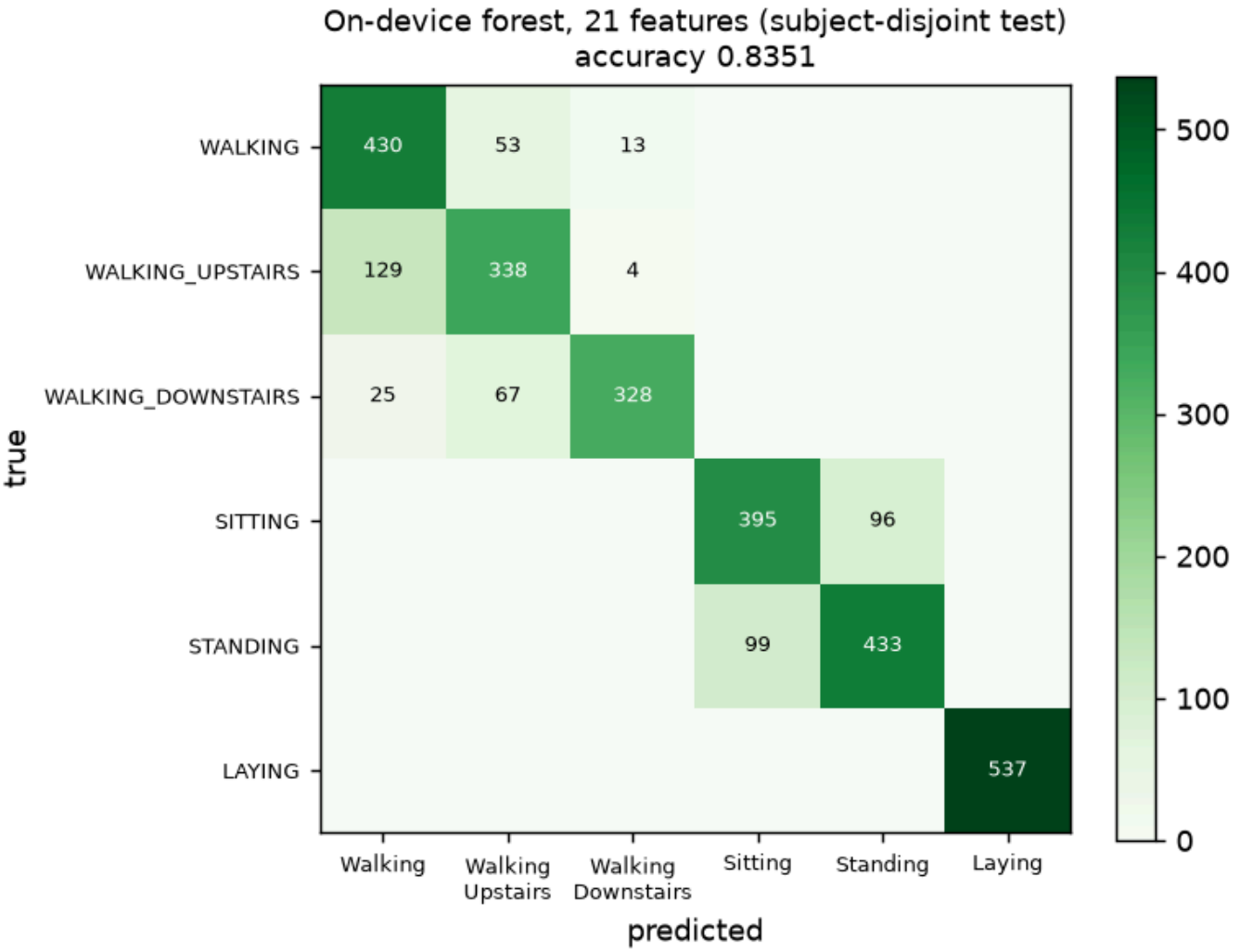
Activity	Precision	Recall	F1	Support
WALKING	0.7363	0.8669	0.7963	496
WALKING_UPSTAIRS	0.7380	0.7176	0.7277	471
WALKING_DOWNSTAIRS	0.9507	0.7810	0.8575	420
SITTING	0.7996	0.8045	0.8020	491
STANDING	0.8185	0.8139	0.8162	532
LAYING	1.0000	1.0000	1.0000	537
Macro average	**0.8405**	**0.8307**	**0.8333**	2947

Where it fails, and why it does not matter much for this app: the three walking variants are confused with each other (upstairs recall 0.7176, and WALKING's precision is only 0.7363 because downstairs windows are called walking). `ActivityKind.isAmbulating` groups all three, and every decision the app makes from activity — "is this person moving?" — uses that grouping. The walking-versus-walking distinction is the least reliable output and also the least used.

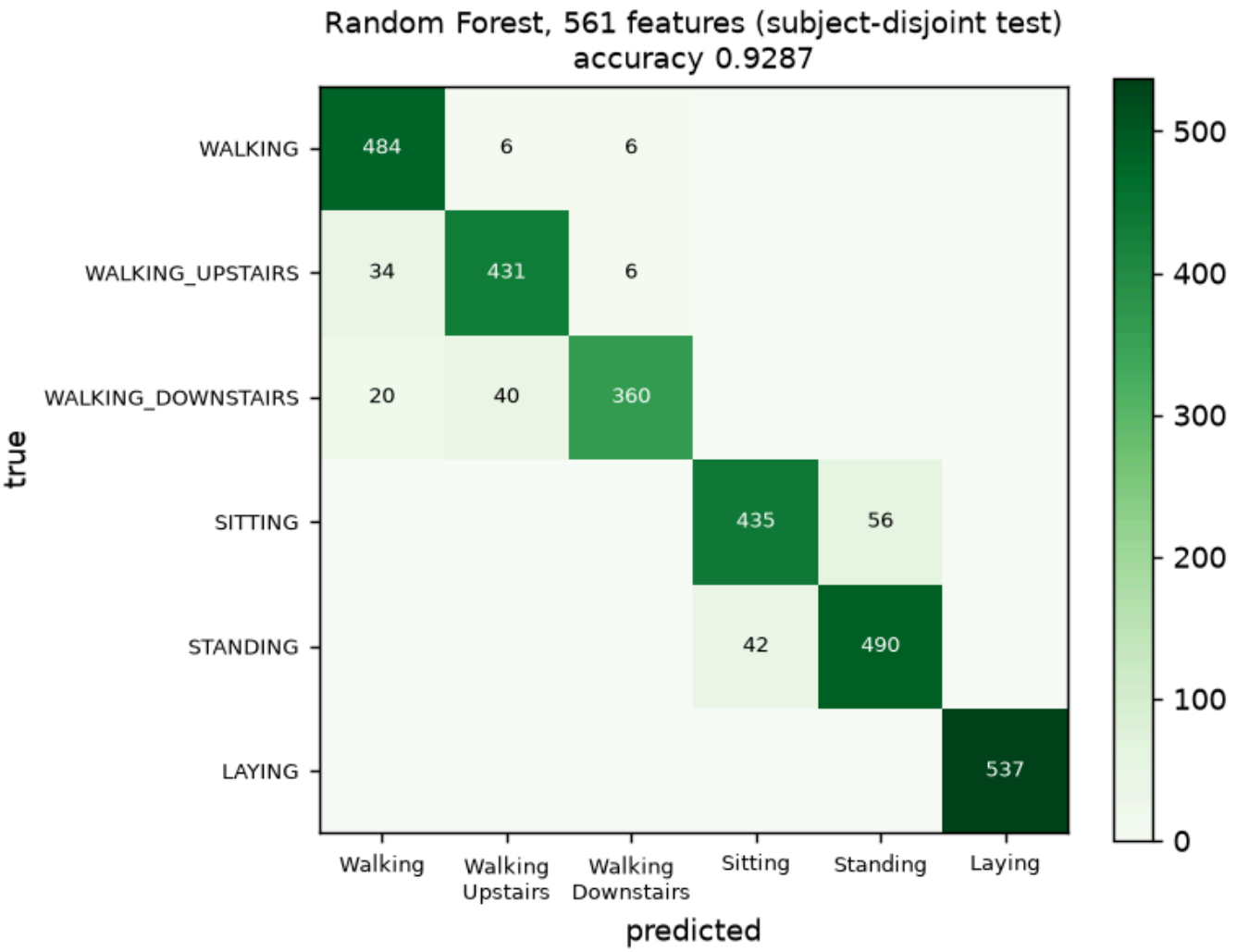
`LAYING` is perfect on this split. That is a genuine property of the data (lying down is kinematically unmistakable) but it is *one* held-out fold of *thirty* subjects, and §8 says what that is worth.

Figures (real, from the training run)

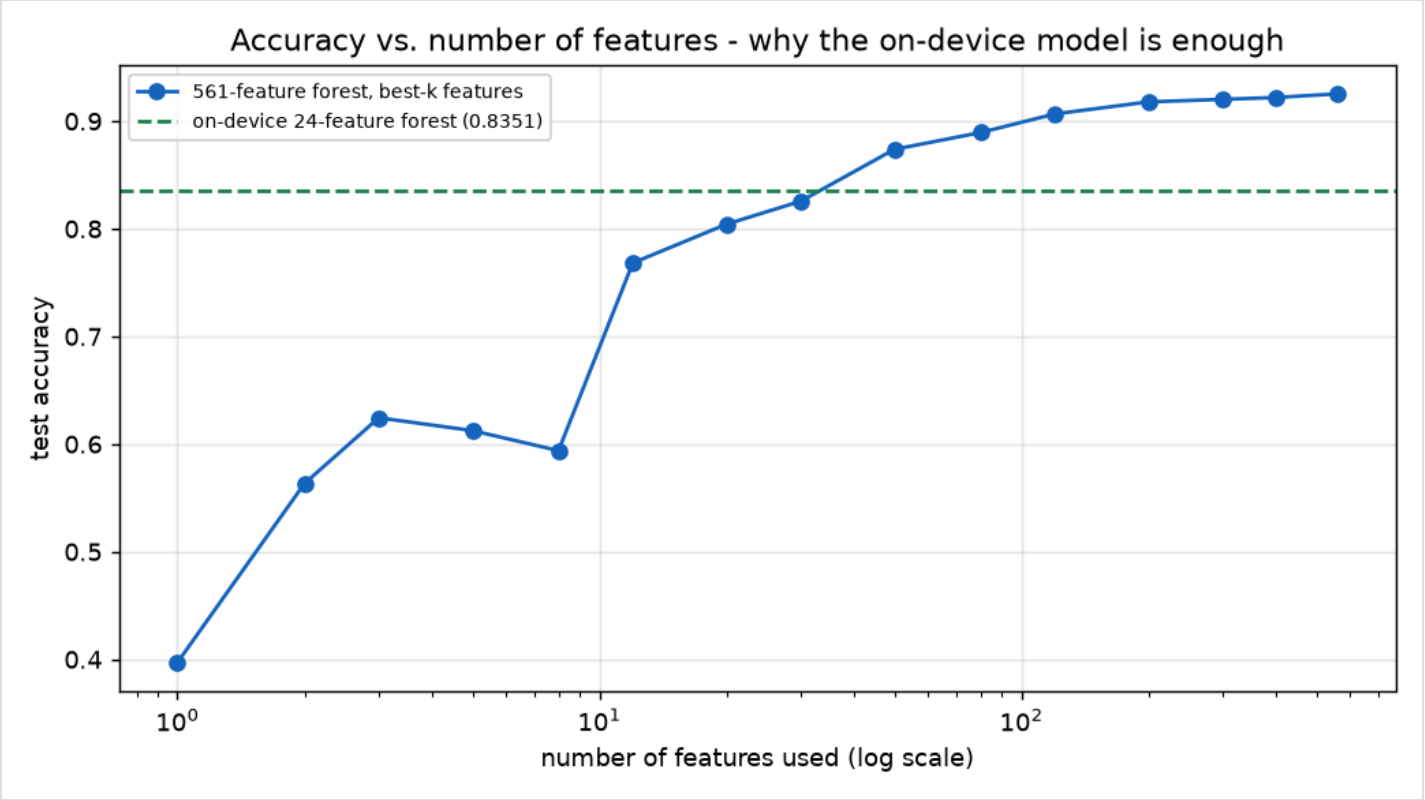
Confusion matrix, shipped 24-feature model (rows = true, columns = predicted):



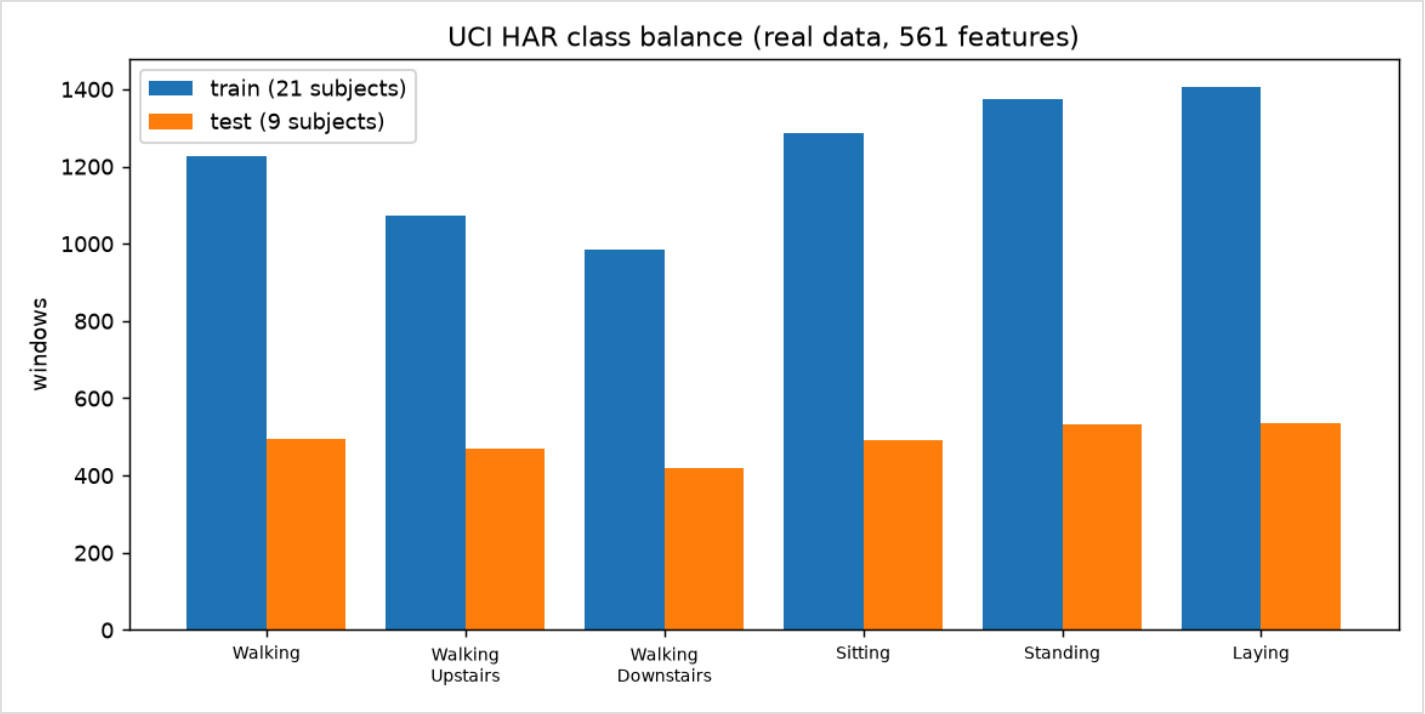
Confusion matrix, 561-feature reference model:

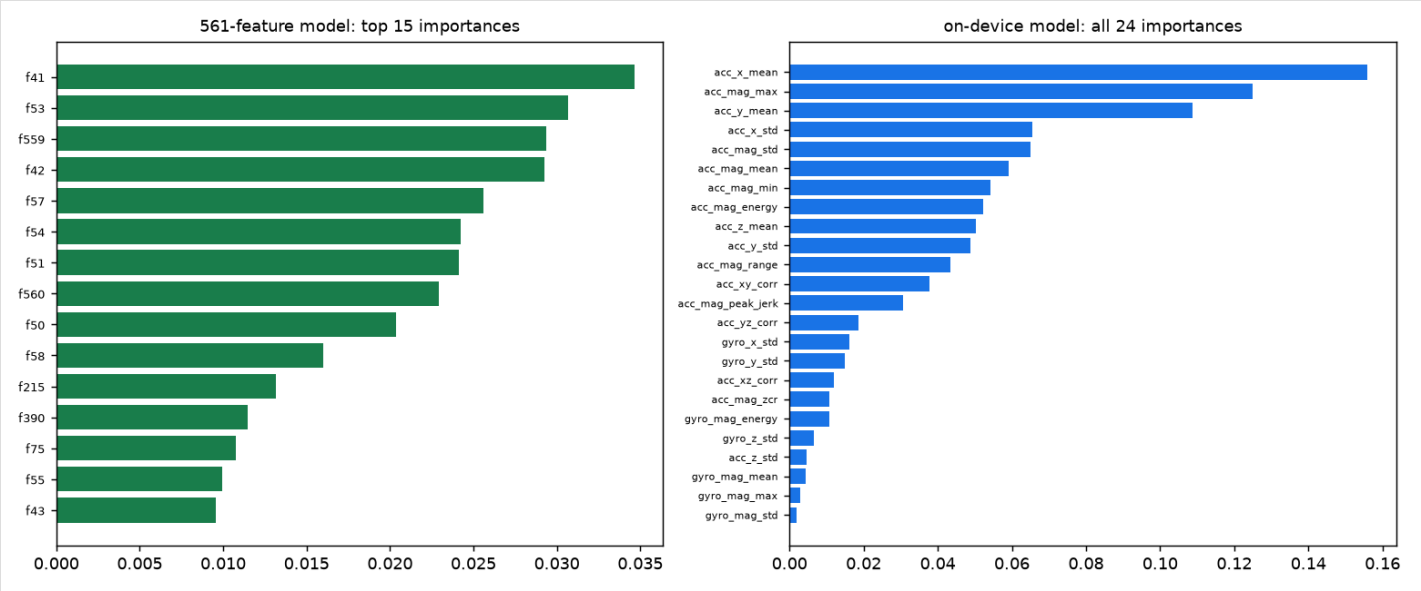


Accuracy versus number of features:



Per-class balance and forest feature importance:





---

5. Model B — impact / fall-signature detector (synthesised positives)

This is the part of the project most likely to be misread, so it is stated plainly and first.

> **UCI HAR contains no falls.** There is no fall class, no "impact" annotation and no subject who fell. So the positive class was **manufactured**: a fall-like acceleration transient (a short free-fall dip, an impact spike of several g, and the stillness that follows) was injected into real recorded windows from the held-out subjects. The negative class is those same recordings untouched.
>
> Therefore the metrics below measure **how well the detector finds the impact signature that was injected**. They are not, and must not be reported as, clinical fall-detection performance.

Configuration: `RandomForestClassifier`, `n_estimators=40`, same 24 features, binary `NO_FALL` / `FALL`.

Split: subject-disjoint, and disjoint from nothing else — the test subjects are 2, 4, 9, 10, 12, 13, 18, 20, 24; the detector trains on the other 21.

Metric	Value
Test windows	1848 (924 positive, 924 negative)
ROC-AUC	**0.9999**
Average precision	**0.9999**

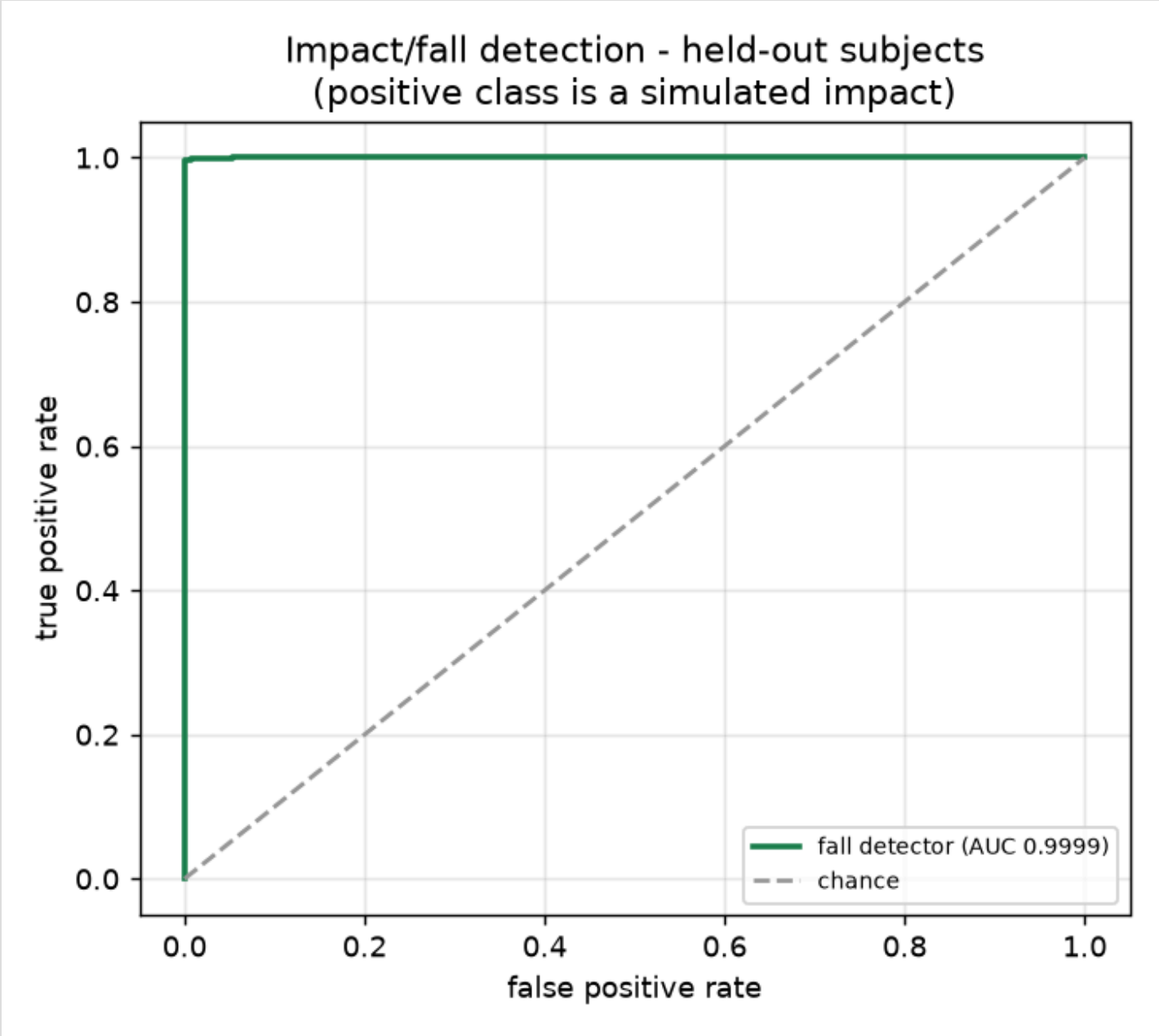
Metric	Value
F1 @ 0.5	**0.9962**
Precision @ 0.5	0.9957
Recall @ 0.5	0.9968
Shuffled-label control (ROC-AUC)	**0.5198**

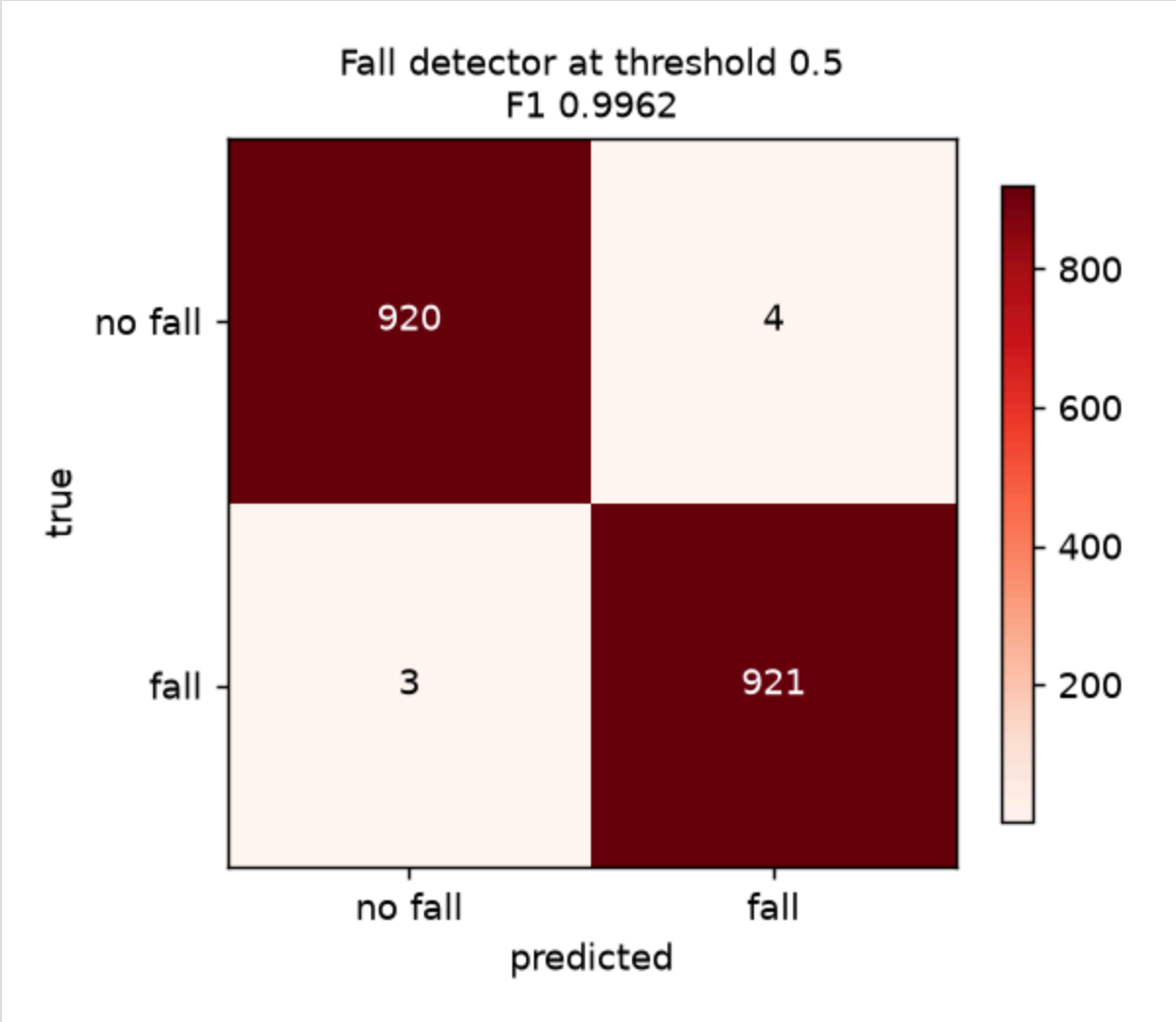
The last row is the one that gives the others meaning. Training the identical pipeline on the same features with the labels randomly permuted collapses ROC-AUC to 0.5198 — indistinguishable from chance. So the near-perfect score above is the detector genuinely finding the injected signature in real held-out subjects' recordings, not an artefact of leakage, duplicated windows, or a feature that secretly encodes the label.

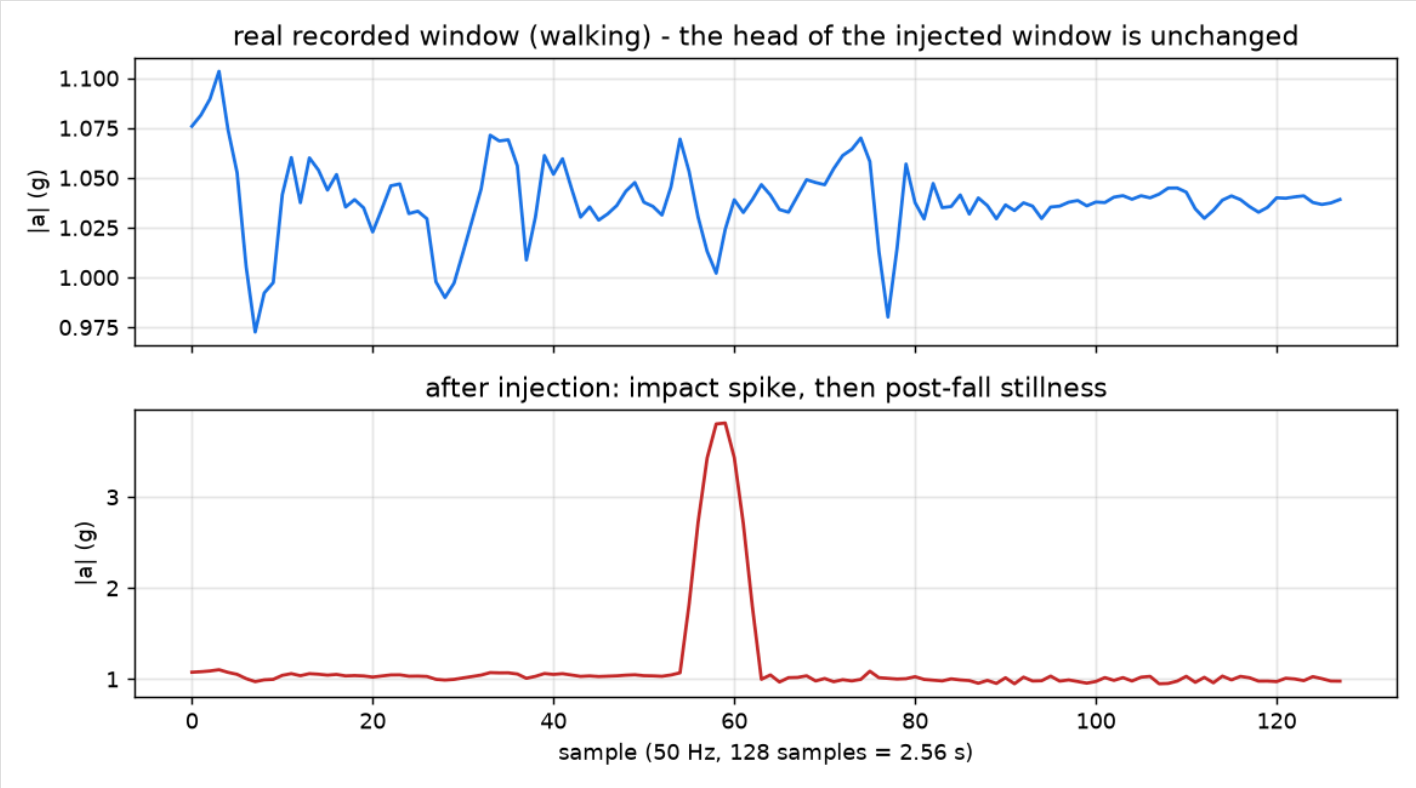
That is still a much easier problem than real fall detection, and it is easy to see why: an injected impact is a large, stereotyped, high-signal transient, and the synthetic positives share a generator. A real fall is messier, sometimes gentle, and often looks like sitting down quickly.

Upgrade path, and it is deliberately named rather than implied: SisFall, UniMiB SHAR and MobiFall are public datasets that contain genuine falls from real volunteers. Retraining on one of those is the single change that would turn these numbers into a fall-detection claim. It has not been done here.

Detector ROC curve, confusion matrix and an example of the injected signal:


Impact/fall detection - held-out subjects
(positive class is a simulated impact)
true positive rate
false positive rate
0.0
0.2
0.4
0.6
0.8
1.0
fall detector (AUC 0.9999)
chance


Fall detector at threshold 0.5
F1 0.9962
no fall
fall
true
920
4
3
921
800
600
400
200
no fall
fall
predicted



---

6. How a fall is decided on the phone

The model's score is **one of three gates**, and the app only ever *suspends* a fall when all three agree. The thresholds live in `FallPolicy` and are published rather than buried:

Gate	Threshold	Why
Impact magnitude	≥ **2.0 g**	a hard fall lands roughly 2–5 g at the waist; below this it is a stumble, a dropped phone or a bang against a rail
Model score	≥ **0.55**	the window looks like the impact signature the detector was trained on
Stillness after the impact	std ≤ **0.12 g**	after a real fall the body stops; the app measures this from the *last* sample at or above the impact threshold, not from the whole plateau
Cooldown	**15 s**	windows overlap by design, so one fall would otherwise raise five alerts
Tail fraction	**0.25**	if the peak lands in the last quarter of the window there is not enough signal left to judge stillness, so that gate is **skipped** rather than failed

Consequences that are visible in the UI: the face shows the probability *and* the threshold, the peak g, the stillness value and its threshold, and it says "suspected". A single high model score with no physical impact never raises anything.

---

7. On-device inference, and the proof it is correct

Export contract

Each tree is exported as five flat parallel arrays (`feature`, `threshold`, `left`, `right`, `value`). Walking a tree is: **go left when** `value <= threshold`, otherwise right; a node with `left[node] == -1` is a leaf. Each tree's leaf counts are normalised to a distribution, then the forest **averages** those distributions — which reproduces scikit-learn's `predict_proba`.

A bug worth recording, because it would have shipped silently: scikit-learn's `tree_.value` holds **class fractions, not counts**. The first exporter wrote `np.rint(tree.value)`, which rounds every node to 0s and 1s and collapses every leaf into a one-hot vector. The model would have loaded, run and answered — just wrongly. It is fixed by exporting `np.rint(tree.value * tree.weighted_n_node_samples)`, and `ml/tests/test_har_models.py::test_exported_values_are_integer_counts_not_rounded_fractions` now fails if the root node of any tree holds implausibly few samples.

Size

File	Bytes	Ships in
`har_activity_model.json`	268 734 (262.4 KB)	`app/src/main/assets/`
`har_fall_model.json`	59 679 (58.3 KB)	`app/src/main/assets/`
`har_activity_model_spec.json`	3 479	`app/src/main/assets/` (documentation asset)

Both models together are under 330 KB, which is the reason a hand-written walker was chosen over a runtime.

Verification that the export is faithful

`ml/har_verify_export.py` re-implements the walker in Python and compares it against scikit-learn on real windows:

Model	Max feature error	Max probability error	Argmax mismatches
Activity	5.458e-07	4.970e-07	0
Impact/fall	6.342e-07	4.736e-07	0

The Kotlin walker is verified independently of Python by `ForestReferenceTest`, which replays 12 real windows per model — with scikit-learn's own probability vectors baked into `app/src/test/resources/har_*_reference_cases.json` — through the shipped JSON assets on the

JVM and asserts the same class probabilities and argmax. So the same model is checked on both sides of the wire, and `ml/tests/test_har_models.py` additionally asserts that the files in `app/src/main/assets` are byte-identical to the verified training output, so "I retrained but forgot to copy it" cannot pass quietly.

---

8. Limitations

1. **No real falls were used.** §5. The strongest limitation in this card.
2. **Lab data.** UCI HAR is 30 volunteers aged 19–48 performing scripted activities with a waist-mounted phone. Real tourists carry the phone wherever, wear bags, and stop to take photos.
3. **6 activities, no fall class.** The activity model's label set is exactly the dataset's; the app does not invent a "falling" activity.
4. **0.8351 accuracy means roughly one window in six is misclassified.** The UI therefore shows the activity *with its confidence* and never acts on activity alone.
5. **Phone placement matters.** Features 9–11 encode the gravity direction, so a phone in a pocket versus held flat gives different means; the model was trained on waist-mounted data.
6. **Nothing here has been validated on a real device by this project.** The build environment for this work had no emulator and no attached phone (`adb devices` was empty), so the sensor pipeline is compile-checked and JVM-tested against recorded windows, but the instrumented cases that would prove it on hardware are recorded as **NOT EXECUTED** in `docs/test_report.md`. That is a gap in evidence, not a passed test.
7. **The approach-risk score shown when nearing a restricted zone is *not* a model.** It is a published weighted checklist (`ApproachRiskModel`) combining zone severity, the tourist's own recorded crossings, distance, time of day and the live movement state. It is deliberately not dressed up as a learned danger probability; see §9 of `docs/final_project_report.md`.

---

9. Reproducing these numbers

```
cd D:\TouristSafetyMonitoring-src
python ml\har_preprocess.py     # extract/verify UCI HAR -> ml\outputs\har\clean\
python ml\har_train.py          # activity model, metrics, figures -> ml\outputs\har\train
python ml\har_fall.py           # injected-impact detector      -> ml\outputs\har\fall\
python ml\har_verify_export.py  # exported JSON vs scikit-learn: expect ALL PASS
python -m pytest ml\tests -q    # 86 tests, includes the artefact contract tests
```

Then copy the artefacts into the app and run the Kotlin side:

```
powershell -ExecutionPolicy Bypass -File .\build.ps1 ":app:testDebugUnitTest"
```

Training is seeded, so the metrics above reproduce exactly; re-running `har_train.py` regenerates files byte-identical to the shipped ones.

---

10. Files

Path	What it is
`ml/har_preprocess.py`	dataset extraction, integrity checks, subject split
`ml/har_train.py`	activity model, per-class report, figures, model + reference-case export
`ml/har_fall.py`	injected-impact generation, detector, shuffled-label control
`ml/har_verify_export.py`	JSON-vs-scikit-learn agreement check
`ml/tests/test_har_models.py`	pytest contract tests for the artefacts above
`ml/outputs/har/clean/`	dataset metadata, class balance, subject split
`ml/outputs/har/train/`	activity reports, metrics CSV, figures, model JSON, spec
`ml/outputs/har/fall/`	detector report, ROC/confusion/signal figures, model JSON
`app/src/main/assets/har_activity_model.json`	shipped activity forest
`app/src/main/assets/har_fall_model.json`	shipped impact detector
`app/src/main/java/com/touristsafety/app/motion/MotionFeatures.kt`	the 24-feature implementation
`app/src/main/java/com/touristsafety/app/motion/DecisionForest.kt`	the JSON forest walker

Path	What it is
`app/src/main/java/com/touristsafety/app/motion/MovementAnalyzer.kt`	window management + the three fall gates
`app/src/test/java/com/touristsafety/app/motion/ForestReferenceTest.kt`	Kotlin-vs-scikit-learn agreement